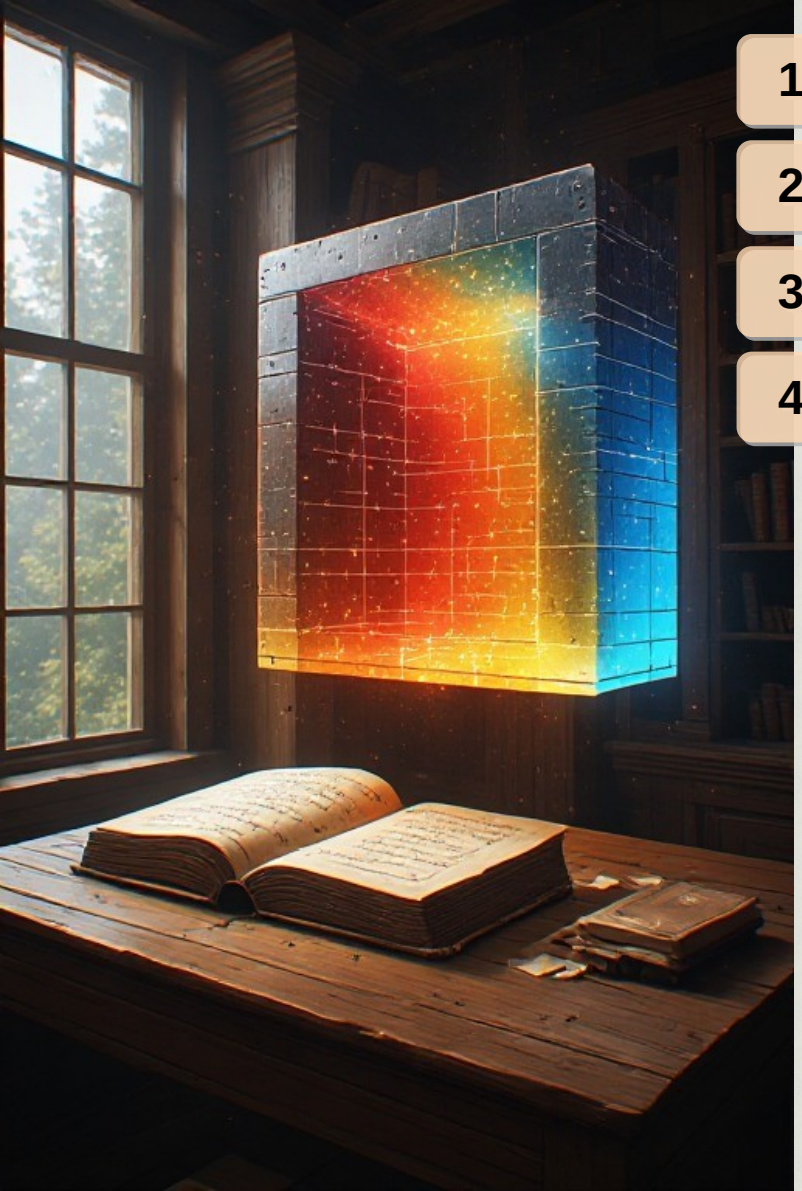
Problema de Difusión de Calor (Ley de Fourier)

Universidad Nacional de La Plata



Facultad de Informática

Prof. Dr. Adrian Pousa



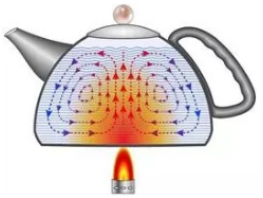
1. El problema de la difusión de calor
2. Algoritmo secuencial
3. Infraestructura del cluster
4. Pautas para la entrega



1 El problema de la difusión de calor



El problema de la difusión de calor



- El problema se conoce como Problema de Difusión de Calor.
  - Simula como se propaga el calor en una superficie aplicando la Ley de Fourier.
- Condiciones:
  - Se tiene una superficie, asumimos dos dimensiones, con determinada temperatura inicial.
  - Se aplica una fuente de calor en determinadas partes de la superficie.
  - El calor se desplaza de las partes mas calientes a las partes mas frías de la superficie.
  - Se simula ese desplazamiento a partir de la Ley de Fourier.
  - En cada paso se calcula, aplicando la Ley, la nueva temperatura de un punto de la superficie en relación con la temperatura de los puntos vecinos.

El problema de la difusión de calor
Ley de Fourier

- La tasa de cambio de temperatura en un punto depende de la diferencia con sus vecinos:

$$\frac{\partial T}{\partial t} = D \nabla^2 T$$

T : *Temperatura*
D : *Coeficiente de difusion termica*
t : *tiempo*
$\nabla^2 T$: *laplaciano de la temperatura* (*suma de segundas derivadas espaciales*)



- **Discretización:** Simulamos la superficie mediante una grilla. Cada celda de la grilla en el tiempo $t+1$ puede representar un punto con una temperatura $T(x,y)$. La ecuación diferencial puede aproximarse:

$$T^{(t+1)}(x,y) = T^{(t)}(x,y) + D[T(x+1,y) + T(x-1,y) + T(x,y+1) + T(x,y-1) - 4T(x,y)]$$



2 Algoritmo secuencial



Algoritmo secuencial
Estructura de programa

- La estructura de programa en lenguaje C sigue un modelo (MCV):

Vista

main.c

```
#include "heat_simulation.h"
…

int main(int argc, char* argv){
      …
      initialize_grid(N);
      …
      for (int step = 0; step < TOTAL_STEPS; step++){
            //Guarda en archivo la grilla actual
            update_simulation();
      }
      destroy__grid();

      return 0;
}
```

Modelo

heat_simulation.h — **Interface**

```
extern float* grid;
extern int grid_size;

void initialize_grid(int N);
void update_simulation();
void mantener_fuentes_de_calor(float* _grid);
void destroy__grid();
```

heat_simulation.c — **Implementación**

```
float diffusion_rate = 0.25f;

void initialize_grid(int N){}
void update_simulation(){}
void mantener_fuentes_de_calor(float* _grid){}
void destroy__grid(){}
```

Compilar:

```
gcc -o heat main.c heat_simulation.c
```

Algoritmo secuencial
Función de simulación - update_simulation( )

- Cada paso de simulación se resuelve en la función `update_simulation()` de la siguiente forma:

Modelo

heat_simulation.c

```
float diffusion_rate = 0.25f;

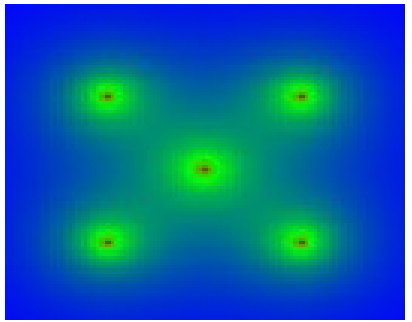
void update_simulation(){
     // Cálculo outplace
          grid(T t): estado anterior
          new_grid(T t+1): nuevo estado

     // Volver a ubicar las fuentes de calor
          mantener_fuentes_de_calor(new_grid);

     // Copiar new_grid en grid para el próximo paso
}
```

$$\frac{\partial T}{\partial t} = D \nabla^2 T$$

$$T^{(t+1)}(x,y) = T^{(t)}(x,y) + D[T(x+1,y) + T(x-1,y) + T(x,y+1) + T(x,y-1) - 4T(x,y)]$$



0º | 25º | 50º | 75º | 100º

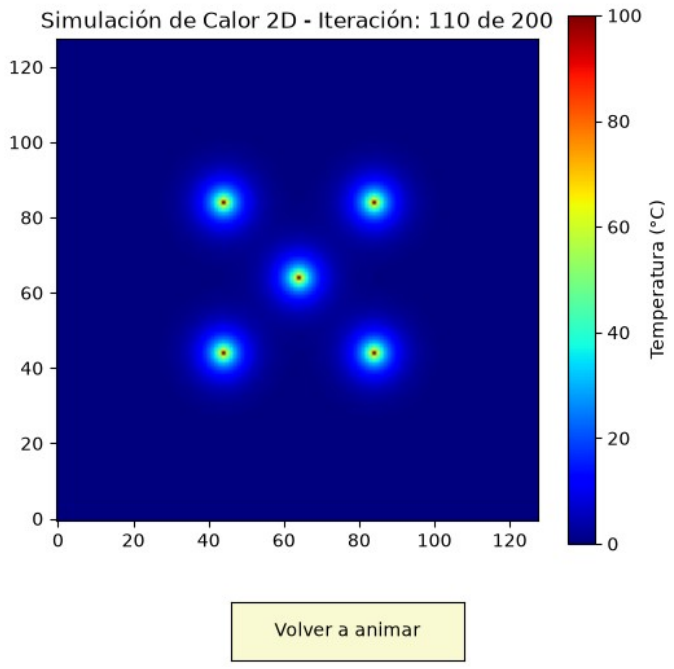
Fuentes de calor

Algoritmo secuencial
Visualización – Script Python `show_blue2red.py`

- Cada grilla se almacena en un directorio `output_frames`
- Para visualizar el resultado utilizamos el *script* Python `show_blue2red.py`:
- El *script* debe estar ubicado fuera del directorio `output_frames`:

```
output_frames
show_blue2red.py

# py show_blue2red.py
```

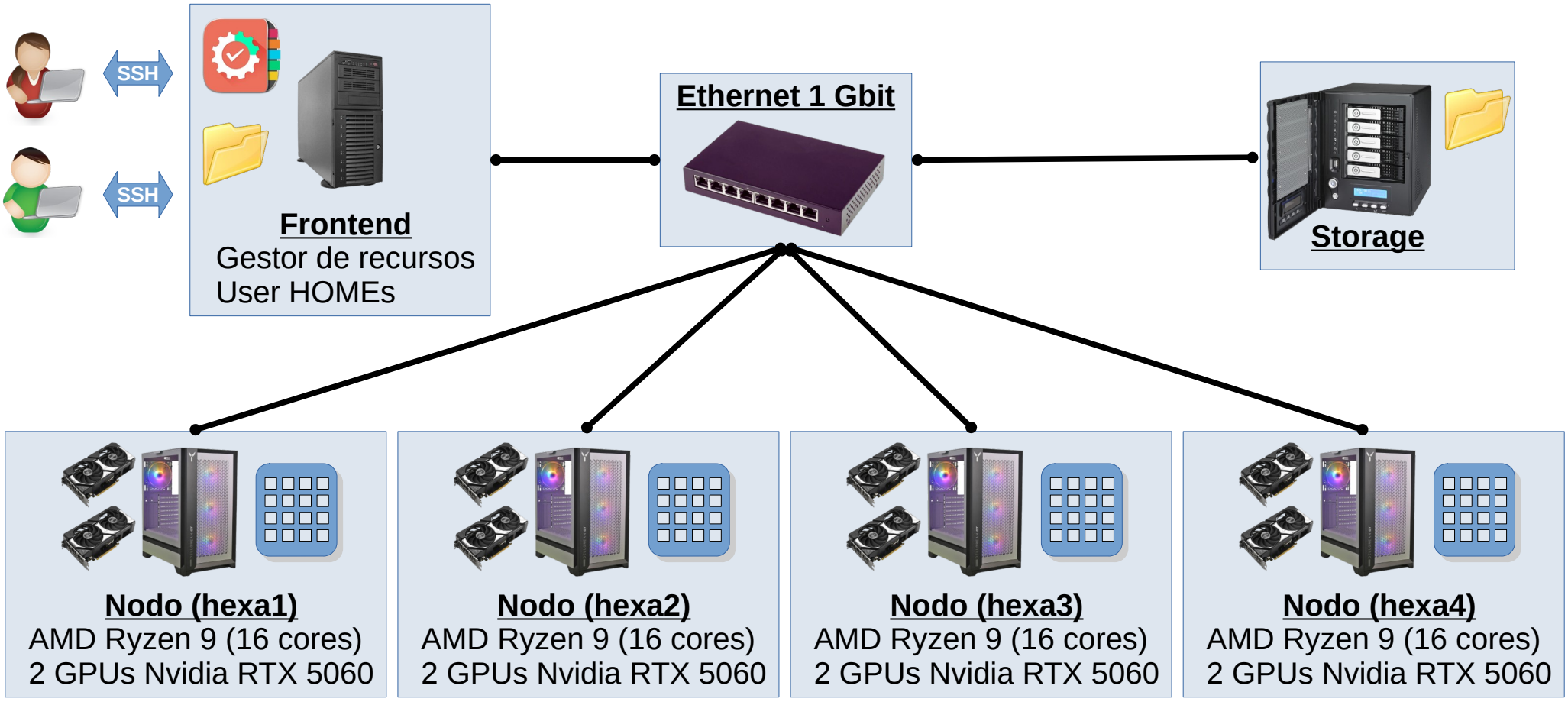




3 Infraestructura del cluster



Infraestructura





4 Pautas para la entrega



Algoritmo secuencial

- Solución secuencial
  - Tamaño de grilla en potencias de 2
  - Número de pasos finito
  - Almacenamiento de cada paso de simulación (Storage) visualización posterior vía script python.
- Solución Paralela un nodo (Parametrizar Block size) (Problema de la frontera)
  - 1 GPU
    - Optimización coalescencia + Memoria Compartida
    - Cuda Stream
  - 2 GPUs (OpenMP)
  - 2 GPUs + AMD cores (OpenMP)
- Solución paralela múltiples nodos
  - MPI + 2 GPUs + AMD cores
  - Master-Worker

Alcance de la entrega

Pasos para el uso de IA (IA-AWARE)

- Usar varios motores de IA y definir el contexto

1) **Analizar el problema** a paralelizar
2) Proponer el **diseño** (estrategia de paralelización): IA para validar y explorar alternativas.
3) **Implementación incremental:** IA para generación rápida de código. Desarrollo incremental paso a paso de acuerdo a las recomendaciones de desarrollo de aplicaciones paralelas.
4) **Analizar el código** generado.
5) Selección de código y **refinamiento**. Comparar distintas soluciones proporcionadas por varios motores de IA. Seleccionar el código mas apropiado y corregir ineficiencias propias de la IA (*NO ES BUENA PARA PARALELISMO*)
6) **Diseño y ejecución** de experimentos: IA para generar *scripts* con variación de parámetros.
7) **Análisis de rendimiento** y comparación de soluciones: análisis de resultados obtenidos. IA para identificar patrones, comportamiento o aspectos ocultos.

